DNSSEC

by

Steven Dormady

An Independent Study Project

Department of Computer Science

James Madison University

Department of Computer Science

April 2026

Contents

Abstract **iii**

1 Introduction **1**

1.1 Motivation 1

1.2 Problem Statement 1

1.3 Contributions 1

1.4 VM Research 2

1.5 VM Network 2

1.6 Bind9 3

2 DNS Setup **4**

2.1 DNS Research 4

2.1.1 DNS Records 5

2.2 The DNS Environment 8

2.2.1 DNS Zones 10

2.3 DNS Vulnerabilities 10

2.3.1 DNS Spoofing/Cache Poisoning 11

2.3.2 DNS Hijacking 11

3 DNSSEC Setup **12**

3.1 DNSSEC Research 12

3.2 New DNS Records 13

3.2.1 Resource Record Set 14
3.2.2 Resource Record Signature 15
3.2.3 DNSKEYs In General 15
3.2.4 Delegation Signer 17
3.3 Chain Of Trust 17
3.4 DNSSEC Options 17
3.4.1 DNSSEC Policy 17
3.4.2 Signing 18
3.5 DNSSEC In The Environment 18

4 Algorithms and Data Structures **19**

A Appendix **21**
A.1 Procedure Manual 21

Abstract

TBD

Introduction

1.1 Motivation

DNS by itself is insecure and vulnerable to attack. With DNSSEC, these vulnerabilities are contained and mitigated.

My personal motivation for this study was a hand on security project, proctored by Dr. Aboutabl, who has become one of my personal favorite professors here at JMU. His Information Security class helped further my interest for security, and when he mentions this study, I knew I had to seize the opportunity.

1.2 Problem Statement

Not sure what to put here, might remove.

1.3 Contributions

Is this necessary?

1.4 VM Research

I have never dealt directly with Virtual Machines before, and building a network of them was something I had never really thought about. However, the thought excited me and it seemed like a great opportunity to learn a new skill. Setting up the first one was a bit of a learning curve, but after setting up the first one to the specifications I wanted, I was able to clone the rest as needed.

The software I used to set up the testbed network for my DNSSEC program was VMWare Workstation. This software made it very easy to set up the virtual machines and network. Having a built in option to create a virtual network was very nice. I was also able to connect to the internet and download the necessary tools I needed, like Bind9, VMWare Tools, Wireshark, gcc and more. I also chose to use git for version control and being able to edit the code on multiple devices if needed.

1.5 VM Network

On VM Workstation, it is very easy and simple to set up a Virtual Network. Under the ”Edit” tab in the menu, I was able to create a new vmnet and customize it to my needs. I made a custom network, isolated from the outside networks. The IP I used was 192.168.107.X, with a subnet mask of 255.255.255.0. The 192.168 is an IP reserved for local networks, so this was perfect for me to to use. There was an option to enable or disable DHCP service. Enabling it meant that you could let DCHP assign addresses to machines on the network. For me, I wanted to statically and manually assign IPs without and DHCP intervention, so I deselected this option to have more control in my environment.

1.6 Bind9

I installed Bind9 on all of the VMs in the Virtual Network. The version I installed was 9.18.39 (check). I decided on Bind9 because it was a good option to be used on Ubuntu machines and gave me the control I wanted.

DNS Setup

2.1 DNS Research

Firstly, I think a bit of background about DNS helps to understand why it became what it is today. DNS started as ARPANET, which was very inefficient and used a centralized hosts file to map the domain names to IP addresses. When a change was made, it had to be redistributed to all of the systems. With number of users and systems needed increasing, this became very hard to maintain, which led to DNS today.[1]

In my research about DNS, I learned a lot about what happens behind the scenes with DNS Servers. Cloudflare called it, ”DNS is the phone book of the internet.”, and I think this name is very fitting.[2] Like a phonebook, you look up a name (domain name) and get someone’s phone number (IP address). All network systems operate with network addresses, such as IPv4 and IPv6. More or less, DNS translates domain names, like www.example.com, to IP addresses, like 134.126.126.99 (jmu.edu), so browsers can load internet resources.

The DNS naming structure is organized hierarchically in a tree-like format. Starting at the root of the tree, there is the Root Domain, which is where real world DNS begins. It delegates authority to the Top-Level Domains, or TLDs, which delegate further to Second-Level Domains, or SLDs, and continue onward, spreading out like a tree. Below in figure 2.1 is a diagram demonstrating this hierarchy.[1]

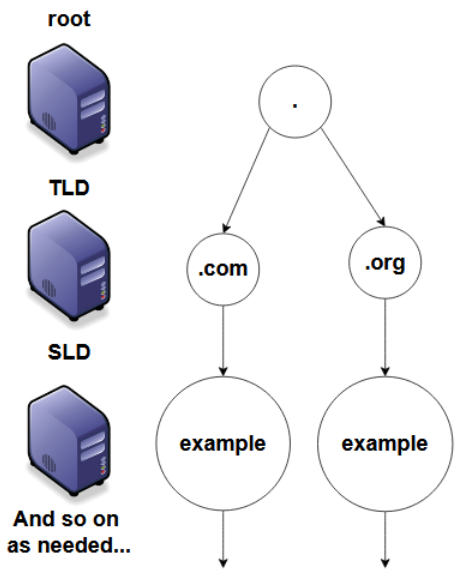


Figure 2.1: Root Delegation Hierarchy

A domain is the label of a node in the tree. A dot in a domain name means that it must traverse to another layer to get more information. Like in the above example, we start with ”example”, being at the Second-Level domain, then traverses to ”.com” at the Top-Level domain, then finally the root, ”.”, server. Put it all together and you get example.com., with the end period making the a Fully Qualified Domain Name, or FDQN. This period is usually hidden behind the scenes, and as users we do not use it. Root servers is the beginning of all DNS name resolutions, which makes the name root make sense. Like a tree, the DNS tree needs it’s roots to work. However, it helps to see the root domain being reached in this instance. There are 13 root servers due to historical reasons of how many addresses could be put into a 512 byte UDP message.[1]

2.1.1 DNS Records

These DNS records are the most common records used in building a DNS server. These records are all in db files, which are database files used for each zone defined. For example, the zone ”.lab” will have a db.lab file. You can name it whatever, as long as it aligns with the named.conf.local zone path defined for the zone. It is best practice to have the domain

name as the db files extension. For both the A and NS records, their line begins with an @ symbol. Below is an example of my db.lab file for the lab zone on VM1.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/bind/db.lab
;
; BIND data file for local loopback interface
;
$TTL    604800
@       IN      SOA     ns1.lab. root.lab. (
                              5         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.lab.

ns1     IN      A       192.168.107.129

jmu.lab.        IN      NS      ns1.jmu.lab.
ns1.jmu.lab.    IN      A       192.168.107.128
bc      IN      A       192.168.107.1
computer        IN      A       192.168.107.2
```

Figure 2.2: The db.lab file for the lab zone.

Start Of Authority Records

SOA records, or ”Start Of Authority” records, show the domain name that the specific db file has authority over. There are several sub records within. The MNAME is the domain name of the name server of the original or primary source of data for the zone. Like a .com, or for this study, .jmu and .lab. These are typically preceded with an ns1, which will come into play later with Namserver Records later. The next record is a RNAME, which is the Responsible Person record. For this project, I have root.jmu or root.lab for the root having responsibility. The next record is the serial number, which Bind uses to load different serial versions of the zone. This record has the space to be an unsigned 32 bit number. It should be updated every time there is a change to the db file. Some developers like to use the date it was edited on to help keep track of when it was last edited. Next, the refresh record is a 32 bit time interval before the zone should be refreshed. The next record is the retry record,

whish is another 32 bit time interval that would run out before a failed refresh should give up. Lastly, the expire record defines another 32 bit time value that gives an upper bound on the time interval that can run before the zone is no longer authoritative. Below is an example of an SOA record for BLANK zone.[3] An example cane be seen below in figure 2.3.

```
@       IN      SOA     ns1.lab. root.lab. (
                              5          ; Serial
                         604800          ; Refresh
                          86400          ; Retry
                        2419200          ; Expire
                         604800 )        ; Negative Cache TTL
```

Figure 2.3: An example of the SOA record in the parent servers db.lab, representing the start of authority for the zone.

Address Records

A records are address records, ”A” for short. These are the records you get when a domain maps to an IP address of a given domain. This is the most fundamental type of DNS record. There are also ”AAAA” records, which are for IPv6 traffic, but for this study we will only focus on IPv4 IPs. These records enable a user’s device to connect with and load a website, or whatever is at the IP address.[4] As seen below, in figure 2.4, when querying for an ip at one of my DNS Servers, I am returned with the IP address of that record.

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 computer.lab A +short
192.168.107.2
```

Figure 2.4: An example of digging for an A record, the short flag used to only give me the ip.

Below, figure 2.5, is an example of what these records look like.

```
bc      IN      A       192.168.107.1
computer        IN      A       192.168.107.2
```

Figure 2.5: An example of A records in the parent servers db.lab, computer.lab and bc.lab

Nameserver Records

NS records stand for ”Nameserver” records, and indicate which DNS server is authoritative for that domain. Like in the SOA, ns1.lab shows which server is authoritative for the .lab domain. Domains can have multiple NS records, like .lab has, which indicate primary and secondary nameservers for domains. For instance, in db.lab, there are multiple NS records, one pointing the the server itself and another pointing to the secondary server for the .jmu.lab domain.[5] See figure 2.6 for an example.

```
@          IN      NS      ns1.lab.

ns1        IN      A       192.168.107.129

jmu.lab.           IN      NS      ns1.jmu.lab.
ns1.jmu.lab.       IN      A       192.168.107.128
```

Figure 2.6: An example of NS records in the parent servers db.lab

2.2 The DNS Environment

There are numerous similarities between DNS with the internet and this testbed study. The bridge between the user and DNS servers is the DNS resolver or name resolution. In the project, the client digs at a server, and gets a response. This is a higher level abstraction, as there are no websites mapped to the IPs on my DNS servers, just more local IPs which could be a website. There is also no official ”.” or root domain, but my ”root” domain is lab., which is also my top level domain. This is seen in figure 2.7. In this setup, authority is held in the parent vm and passed downward onto the child server. In real DNS, a recursive resolver is used and traverses the different levels of domains.

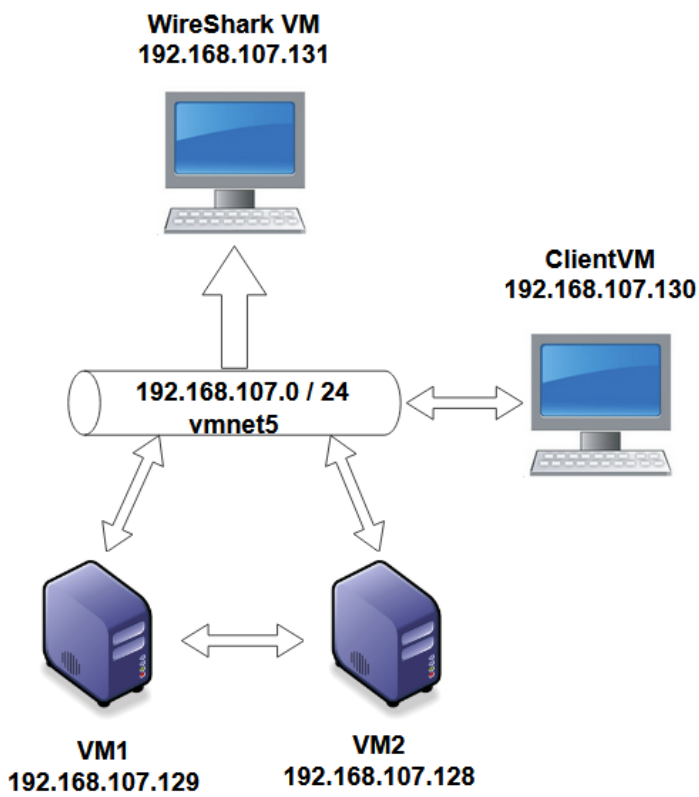


Figure 2.7: This is a diagram of the setup of the virtual environment, with inspiration of the layout coming from Dr.Aboutabl

In this study, both the parent and the child server have authority over their own domains, and you can also traverse the tree to query the parent for information on the child server, as seen below in figure 2.8. There are also no IPV6 records, and this is disabled in the configuration of the servers.

```
;; AUTHORITY SECTION:
jmu.lab.                86400   IN      NS      ns.jmu.lab.

;; ADDITIONAL SECTION:
ns.jmu.lab.             86400   IN      A       192.168.107.128

;; Query time: 1 msec
;; SERVER: 192.168.107.129#53(192.168.107.129) (UDP)
;; WHEN: Tue Feb 24 22:57:16 EST 2026
;; MSG SIZE  rcvd: 102

steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.128 quad.jmu.lab A +short
192.168.107.64
```

Figure 2.8: Querying the parent for information about the child, the querying the child with that information.

2.2.1 DNS Zones

In the environment, there are different zones between the two name servers. As seen in figure 2.9, the parent zone holds .lab, with an extension to the child zone, .jmu. To be able to query the child, you need a domain name, followed by .jmu.lab. A full example of a valid query would be like this: quad.jmu.lab. Quad is the final piece, and this domain maps to 192.168.107.64. You can query the child server directly and receive responses, or you can query it through the parent server.

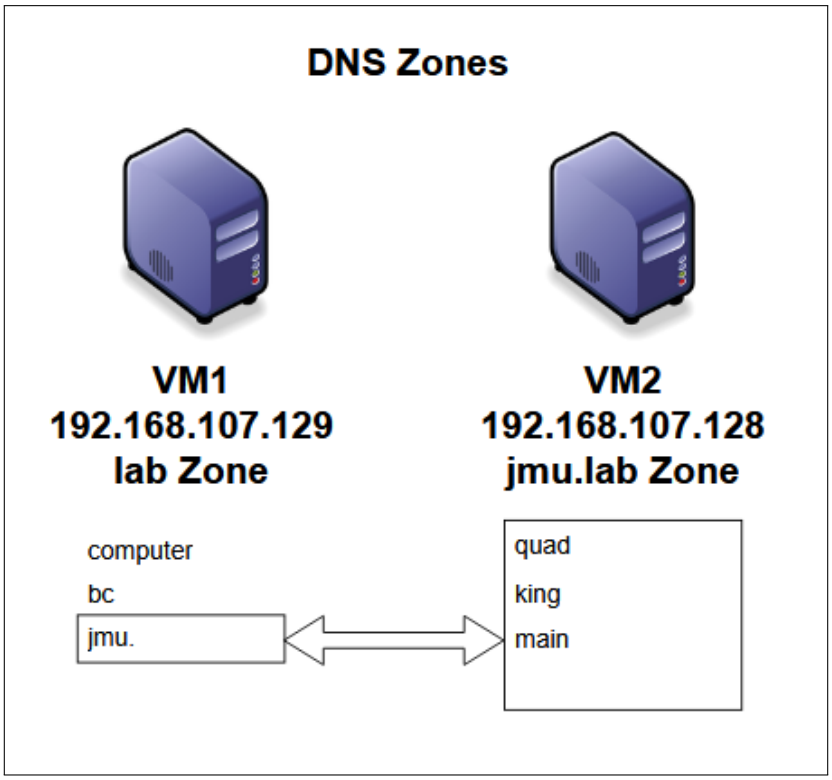


Figure 2.9: This is a diagram showing the zones and their relationship.

2.3 DNS Vulnerabilities

DNS without DNSSEC is very insecure and vulnerable to a plethora of attacks. Without DNSSEC, queries are readable to any person on the internet. An attacker could easily intercept this traffic and replace it with malicious content. This is where DNSSEC comes

into play, helping to mitigate and prevent many attacks from occurring. It also proves that the information is coming from a trusted, authoritative source.

2.3.1 DNS Spoofing/Cache Poisoning

This is an especially vulnerable part of DNS without DNSSEC. This attack is a textbook man-in-the-middle attack, when the attacked intercepts a query and replaces it with their malicious information. DNS spoofing is when an attacker inserts their malicious address in place of a cached address. This attack is using DNS Hijacking in this instance, to redirect to what the attacker wants them to see. They pretend to be a DNS nameserver and forge a reply, and with UDP it makes it even easier for this attack to occur. Cache poisoning is DNS Spoofing without having to hijack the query. The systems involved with DNS, like servers, computers, and routers can cache DNS lookups for quicker recall. An attacker can poison the cache by replacing an entry with a spoofed domain. This spoofed domain gets accessed until the cache gets refreshed and the spoofed site gets replaced.[6]

2.3.2 DNS Hijacking

DNS Hijacking occurs when an attacker tries to redirect where the client is going with their query. The can perform a man-in-the-middle attack and take the query and replace it with their IP, causing the user's machine to directly communicate with the attacker. They could also give an IP that delivers a malicious payload. These are some of the most basic types of attacks when it comes the DNS hijacking.[7]

DNSSEC Setup

3.1 DNSSEC Research

DNSSEC, or Domain Name System Security Extension, adds a layer of security to the otherwise vulnerable DNS. DNSSEC adds security by using cryptographic signatures with public key cryptography, with the private key signing it and the public key verifying. DNSSEC also implements hashing, which helps to add more confusion and making it harder for attackers to know what plain text strings were previously used. It also adds data integrity and and authentication to the data, showing whether it was tampered with and the signing provides the authentication.

There is an additional flag with dnssec queries, the ”ad” flag. This flag is passed along with successful DNSSEC queries, meaning ”Authenticated Data”. When you query for a domain with dnssec enabled, receiving this flag back instead of a SERVFAIL status means that the data was successfully authenticated. You will not see it in the below screen shots, and that is because there isn't any validation going on yet. Seeing the records is enough for this part of my project, where as the next chapter I will write code to validate these records.

Below, figure 3.1, is db.lab, but now set for DNSSEC. It changed directories to allow for permission to be more for the owner of the server (me). Originally, DNS configuration and the db files were in /etc/bind/, but now they moved to /var/lib/bind/ to allow for more freedom and not having to deal with bind ownership. Changing the zone configuration to

point to the new db file allows the server to write into this directory. As you can see below, there is a good number of changes to the db file. Gone are the @ symbols preceding lines, and there is a new minimum time record. There is still a SOA and NS and A records, which have minimally changed from regular DNS. At the bottom of the figure, yuo can see a Delegation Signer record, which came from the child zone. This is an important part of the Chain of trust, allowing the parent to verify the child is the child.[8]

```
$TTL    86400
lab.    IN      SOA     lab. root.lab. (
                              3         ; Serial
                          43200         ; Refresh
                            900         ; Retry
                        1814400         ; Expire
                           7200 )       ; minimum

lab.    IN      NS      ns.lab.
ns      IN      A       192.168.107.129

jmu     IN      NS      ns.jmu.lab.
ns.jmu  IN      A       192.168.107.128
; bc    IN      A       192.168.107.1
computer        IN      A       192.168.107.2
jmu IN DS 34939 8 2 9682051422A73107F882F98D04F47492ECA60648DC071428E43477AC7F1
42C2
```

Figure 3.1: The new DNSSEC db.lab

3.2 New DNS Records

With the regular DNS records previously discussed, there are some new records that must be taken into account. Regarding the Zone Signing Key and the Key Signing Key, the algorithms I used for the keys was RSASHA256, this is a good algorithm for encryption and one I am familiar with. The ZSK signs the records from the zone, and the KSK signs the ZSK. The hand shake looks a bit like this, an example of a query for computer.lab, figure 3.2.

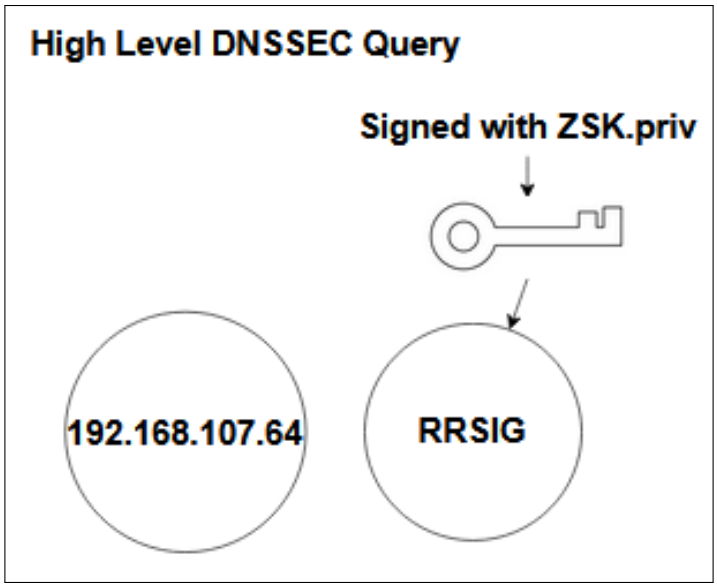


Figure 3.2: A diagram showing a high level view of a DNSSEC query, digging for an Address Record.

3.2.1 Resource Record Set

This record is not an explicit record, rather a descriptive word to describe records that are grouped together. For instance, if I query for my parent server for DNSKEYs, I receive the RRSet that is signed, seen below in figure 3.3. These keys are signed together as a set by the Key Signing Key, which the name is pretty explanatory for what it's purpose is. [9]

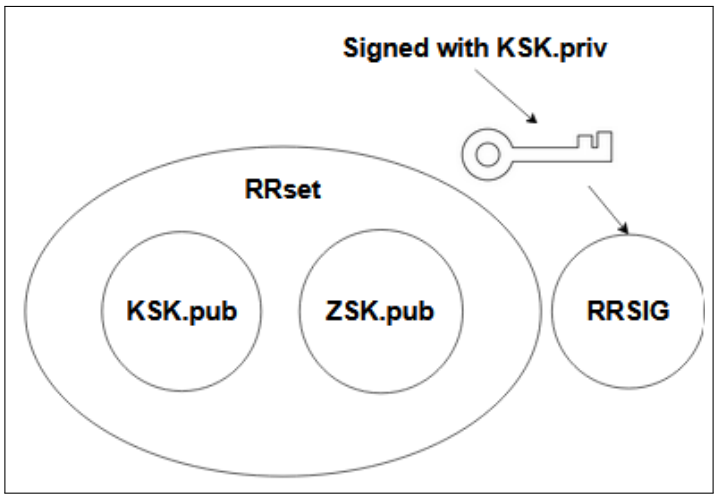


Figure 3.3: This query shows a dig at the parent for the DNSKEYs

3.2.2 Resource Record Signature

The Resource Record Signature, or RRSIG record, is used to verify the answers that are received. These are attached to every answer that is sent with DNSSEC. The signatures are used by recursive name servers to verify that the answer to the question is valid. [10] Below is an example of an address query with a signature, figure 3.4.

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 computer.lab +dnssec +multiline

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.107.129 computer.lab +dnssec +multiline
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 19718
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 3df3cdc0dcb579af0100000069a5d721e88ca1e9eeb51c37 (good)
;; QUESTION SECTION:
;computer.lab.          IN A

;; ANSWER SECTION:
computer.lab.           86400 IN A 192.168.107.2
computer.lab.           86400 IN RRSIG A 8 2 86400 (
                                20260310233437 20260225023500 34104 lab.
                                GkosYckWQnxLuVJhdzymzDDRZuS+hBs5zzsdmrBCFkc3
                                4hdxuO8bbGjPQfkJTB9fXyov9xc6M1rDQko8+qzgD+9d
                                Kuextffp+oU3kiOLiJosuEUlDciTkVR/u4nm0T84STDN
                                RmK1yjPtp+hb4ZK0eLTIm3Ksqx/xQsT6Pl6Gl/Q= )

;; Query time: 0 msec
;; SERVER: 192.168.107.129#53(192.168.107.129) (UDP)
;; WHEN: Mon Mar 02 13:29:53 EST 2026
;; MSG SIZE  rcvd: 248
```

Figure 3.4: Address record with a RRSIG from the ZSK

3.2.3 DNSKEYs In General

DNSKEYs are the record that are used to sign an validate RRsets. These keys use public key cryptography, storing the public keys in DNSKEY RRs. These keys are used in different ways, and there are two different keys that we care about in this study, which are the Zone Signing Key and the Key Signing Key. The identifying type number that this RR is a DNSKEY is 48.

The general format for these RRs is as fo[9] figure 3.5

```
;; QUESTION SECTION:
;lab.                    IN DNSKEY

;; ANSWER SECTION:
lab.                    3600 IN DNSKEY 257 3 8 (
                                AwEAAdyubupVYO97+QR5LxX3eIh8vxOI5bxfVz6ieZqM
                                F19e5Ul2SD+MyyY5iM1VRutYEpLmfbMDDuW3ZElJzN4B
                                RP8huMuzRf8V4nAuB3poxIUs+kRywr4n6et46oCBF7oW
                                mcw792jAKKXFiuegwZ3s5NvrZufCckq5wzgaPvMxJLZL
                                kalddzkc0KuvwAtmGsOTT7lkn3sx59JgWGSYiL+3FNZF
                                SZPlcDyw3QuIO1hN+Vq2NwuSR2LkOFX3miCKx4d07hr2
                                7TtP3LGJiBhcyMeJRT3i4bOBbCtwGelA6xq5i4k1fg2+
                                R2e+2VZtqk35GJsh8c0yqKoahz96ZCSrWCHm690=
                                ) ; KSK; alg = RSASHA256 ; key id = 5852
lab.                    3600 IN DNSKEY 256 3 8 (
                                AwEAAbDODYwllXwQTD9OaDxw02d579560Yf4dGH7VpOb
                                +FcTTMIovHAAPuSgD0l5b1DnxWVhqupG9rM8b6FKxC+6
                                zNJycr9nUjTLkBxaVYgicGaS+X/0qy0K+d7spTg4qw3W
                                jzAls4LqquwljeMTNLe/peC2BZM/cmz0Mhn1MacYqqvH
                                ) ; ZSK; alg = RSASHA256 ; key id = 34104
lab.                    3600 IN RRSIG DNSKEY 8 1 3600 (
                                20260312172022 20260226164849 5852 lab.
                                sTOzJDy11sodWrqEb63MmZzOwhEfgO1ve7n/YgjL1ooQ
                                ECvhsNfYXM5D8Dq2R//oatu17DR9gT2fM591t/Hgi8lI
                                9B69/iPGOvArrKiK3PqmCRFjBnhl5T7ARcsvmA6Yedlj
                                yO2YitOs3aHYIZPoFhmRWrX0OwRA6rVO7/1JvdY/sZDI
                                Odi/oekocHkJ2pBdHP4tymB5nAiSn1Zm2RkGgNSJeQ1w
                                PvGccqXZIDuudTtI6Dy1+JTBZffvpYKwHupy3pCJJTdj
                                DNQ9rpsS15GlignoH0fX4B1WmhvHt5CHFWm5/tDyrXYh
                                XIsiOP9XSx6X8RojnKATV+3DilfgbMgXnQ== )

;; Query time: 0 msec
;; SERVER: 192.168.107.129#53(192.168.107.129) (UDP)
```

Figure 3.5: This query shows a dig at the parent for the DNSKEYs

Zone Signing Key

The Zone Signing Key, or ”ZSK” for short, is the key that is used to sign data from a zone. It signs all records except for RRset that are related to keys. The ZSK has the 256 flag, indicating that it is the ZSK.

The keys follow a public key encryption scheme, with the zone keeping the private key and the public being sent to the user. They use this key to then verify the signature and receive the zone data.[11]

This key has a shorter Time To Live, or TTL, because it is sent out and used much more frequently than the KSK.

Key Signing Key

The Key Signing Key, or ”KSK”, is used to sign what the ZSK does not sign: key-related RRsets. This keys serves another purpose, as it is the bridge between a parent and a child zone. The flag for the KSK is 257, indicating that this key is the KSK. Like the Zone Key, this key also follows the public key encryption scheme. This key pair is meant to protect the ZSK for the zone being queried.

3.2.4 Delegation Signer

The Delegation Signer Record, or DS record, is used to ”vouch” for another zone. The parent holds a copy of this in their zone, and when asked to query about the child zone, it can verify that the response comes from the child.

3.3 Chain Of Trust

The Chain of Trust is how different levels of servers know they can trust each other. There must be trust through the whole process of the query, otherwise there is a vulnerable spot for intrusion by an attacker.[12] An example of a trust anchor is below in figure 3.6.

```
trust-anchors {
        lab. static-key 257 3 8 "AwEAAdyubupVYO97+QR5LxX3eIh8vxOI5bxfVz6ieZqMF19e
5Ul2SD+MyyY5iM1VRutYEpLmfbMDDuW3ZElJzN4BRP8huMuzRf8V4nAuB3poxIUs+kRywr4n6et46oCBF
7oWmcw792jAKKXFiuegwZ3s5NvrZufCckq5wzgaPvMxJLZLkalddzkc0KuvwAtmGsOTT7lkn3sx59JgWG
SYiL+3FNZFSZPlcDyw3QuIO1hN+Vq2NwuSR2LkOFX3miCKx4d07hr27TtP3LGJiBhcyMeJRT3i4bOBbCt
wGelA6xq5i4k1fg2+R2e+2VZtqk35GJsh8c0yqKoahz96ZCSrWCHm690=";
};
```

Figure 3.6: An Example of my Trust Anchor

3.4 DNSSEC Options

Section to talk about my dnssec policy, key management, etc.

My client is the validator, with dnssec-validation set to yes. It has a trust anchor that was manually copied from the parent zone.

3.4.1 DNSSEC Policy

I adjusted the settings in the options configuration file, mainly to specify what algorithm to use, but also to set specifics for the keys. This is seen below in figure 3.7.

```
dnssec-policy rsa-policy {
        keys {
                ksk lifetime 365d algorithm RSASHA256 2048;
                zsk lifetime 60d algorithm RSASHA256 1024;
        };
};
```

Figure 3.7: This is what I added to my dnssec policy I created called rsa-policy

3.4.2 Signing

Right now, I am using Inline Signing, which automatically generates keys and signs the files. This setting also creates and keeps a separate signed zone file.

NOTE: Still gathering information on this, and seeing if fully manual signing is worth it, because I'm not sure if you can sign RR independently

3.5 DNSSEC In The Environment

With DNSSEC on the internet, there is normally a validating resolver separate from the client machine that authenticates and validates the answers from the server before the client gets the response back. Normally, this is done recursively. For this study, there is no recursion and instead the answers will be manually validated.

Algorithms and Data Structures

Bibliography

[1] BIND9, "Introduction to dns and bind 9." https://bind9.readthedocs.io/en/v9.18.39/chapter1.html#the-domain-name-system-dns, 2025. Accessed: 2026-1-1.

[2] Cloudflare Learning, "What is dns?." https://www.cloudflare.com/learning/dns/what-is-dns/, 2025. Accessed: 2025-12-22.

[3] P. Mockapetris, "Domain names - implementation and specification." https://www.rfc-editor.org/rfc/rfc1035, 1987. Accessed: 2026-1-1.

[4] Cloudflare Learning, "Dns a record." https://www.cloudflare.com/learning/dns/dns-records/dns-a-record/, 2025. Accessed: 2026-1-20.

[5] Cloudflare Learning, "Dns ns record." https://www.cloudflare.com/learning/dns/dns-records/dns-ns-record/, 2025. Accessed: 2026-1-20.

[6] Cloudflare Learning, "What is dns cache poisoning? — dns spoofing." https://www.cloudflare.com/learning/dns/dns-cache-poisoning/, 2025. Accessed: 2026-2-22.

[7] Imperva, "What is dns hijacking / redirection attack." https://www.imperva.com/learn/application-security/dns-hijacking-redirection/, 2026. Accessed: 2026-2-26.

[8] Ubuntu Server, "Installing dns security extensions." https://documentation.ubuntu.com/server/how-to/networking/install-dnssec/, 2026. Accessed: 2026-2-20.

[9] R. Arends, et. al, "Resource records for the dns security extensions." https://datatracker.ietf.org/doc/html/rfc4034, 2005. Accessed: 2026-2-23.

[10] BIND9, "Dnssec guide." https://bind9.readthedocs.io/en/v9.18.39/dnssec-guide.html, 2025. Accessed: 2026-2-25.

[11] BIND9, "Introduction to dns and bind 9." https://bind9.readthedocs.io/en/v9.18.39/chapter5.html#zone-keys, 2025. Accessed: 2026-2-25.

[12] BIND9, "Trust anchor." https://bind9.readthedocs.io/en/v9.18.39/dnssec-guide.html#trust-anchors, 2025. Accessed: 2026-2-25.

Appendix

A.1 Procedure Manual

Steps:

1. Creating a VM
    a. First, I created a VM on VMWorkstation. I selected custom setup and kept most of the settings default. My method for adding the OS to the machine was to download the LTS version of Ubuntu to my desktop and using an ISO image, I copied it onto the machine. You can create an account for your machine and name your machine whatever you may like. I gave my vm 2gb of ram and 20 gb of disk (disk space is a few steps later), as this was the recommended minimum for Ubuntu.
    b. FOR THE TIME BEING, use a NAT network condition so you can download necessary things from the internet. The default options till the end is fine, using the recommended options is something I would recommend too.
    c. Resizing VM Disk space: I started with 20gb of disk, enough to comfortably install ubuntu and have some room to work. I wanted the vm to have 40 gb of disk space to have room for software and programs I would need to implement DNSSEC later. It is probably best to backup or take a snapshot of your vm before doing this step.
        i. Turn off the machine. Expand the disk in the VM settings, with expand disk to the desired size. Then, restart the machine and follow the commands below.
        ii. For commands, I ran:
            1. *lsblk* - I ran this to see where my root partition was, like /dev/sda1 or whichever says 20G. In the picture below, it is sda2, originally 20G, now 40G after resizing.

```
steve@steve-VMware-Virtual-Platform:~$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0  73.9M  1 loop /snap/core22/2045
loop2    7:2    0 251.6M  1 loop /snap/firefox/7672
loop3    7:3    0  73.9M  1 loop /snap/core22/2216
loop4    7:4    0 251.7M  1 loop /snap/firefox/7720
loop5    7:5    0  11.1M  1 loop /snap/firmware-updater/167
loop6    7:6    0  18.5M  1 loop /snap/firmware-updater/210
loop7    7:7    0   516M  1 loop /snap/gnome-42-2204/202
loop8    7:8    0 531.4M  1 loop /snap/gnome-42-2204/247
loop9    7:9    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop10   7:10   0  49.3M  1 loop /snap/snapd/24792
loop11   7:11   0  10.8M  1 loop /snap/snap-store/1270
loop12   7:12   0   576K  1 loop /snap/snapd-desktop-integration/315
loop13   7:13   0  48.1M  1 loop /snap/snapd/25935
loop14   7:14   0   576K  1 loop /snap/snapd-desktop-integration/343
sda      8:0    0    40G  0 disk
├─sda1   8:1    0     1M  0 part
└─sda2   8:2    0    40G  0 part /
sr0     11:0    1   5.9G  0 rom  /media/steve/Ubuntu 24.04.3 LTS amd64
```

2. Next, ensure that you have cloud guest tools installed: *sudo apt-get update* and *sudo apt-get install cloud-guest-utils*
3. Next, expand the partition with *sudo growpart /dev/sda 1*, change the number to the partion number shown by lsblk.
4. Then run *sudo resize2fs /dev/sda<previous partion number here>* .
5. The run df -h to verify the new size

d. After reportioning, we can download some software before switching to a virtual network. For ubuntu, I downloaded BIND9 to set up my DNS Servers
   i. Command: *sudo apt install bind9* and *sudo apt install dnsutils*
      1. DNSutils provides helpful troubleshooting and debugging tools, and may not be installed with bind9

e. Also, install gcc to run c programs you might need later. If you would like to as well, install Wireshark on a VM as well to monitor and see the traffic you are sending.

f. VMTOOLS
   i. I tried numerous trouble shooting techniques to install VM Tools on the vm, but I ended up doing it through the command line instead of VMWare. I did not have administrative privileges on the machine and I think that is what prevented me from doing it through VMWare.
   ii. To install them, make sure you have an internet connection and run :
      1. *sudo apt-get install open-vm-tools-desktop*. Says yes when prompted, and once complete check the status with: *systemctl status vmtoolsd*. You should see "active" in green text.

g. At this point, cloning once for another machine to treat as a dns server is not the worst idea, I made the mistake of cloning a bit too early so I just had to repeat steps on multiple vms. It wouldn't hurt either to set up the virtual network and make sure your virtual network can be accessed by the virtual machine.

2. Setting Up the Virtual Network

a. Under the "Edit" Tab in the menu bar, there should be a Virtual Network Editor Option. Select this to begin setting up the network.

b. Select the "Add Network Option to create your new network.

c. Make the vmnet# one that is not taken yet. For me, 0, 1, and 8 were in use, so I used vmnet5. Select Host-only for the network type.

d. After selecting add and creating the network, you will be able to change specifics of the network. Enter the virtual machine settings and add a network adapter setting. Deselect DHCP service to distribute IP addresses to VMs. The IP addresses will be statically assigned by you. Connecting a host virtual adapter can also be deselected, it should not be needed to connect with the host on this network.
e. You can manually input a private IP address, like 192.168.x.0, or if you save, the editor will give you a random 192.168.x.0 IP address.

3. Cloning the VM
    a. Ensure the VM you intend to clone is powered off.
        i. Before cloning, ensuring that you have a Network Adapter that is connected to the vmnet you created in the previous step. To do this, navigate to the vm settings by right-clicking on the vm itself, clicking on the vm menu tab, or while it is turned off it can be seen while viewing the vm.
        ii. Under "Hardware", select add at the bottom of the screen. Select Network Adapter and finish.
        iii. Next, select the newly added Network Adapter, likely called Network Adapter 2 if you still have the original NAT connection.
        iv. Select Custom and your vmnet# should be available to choose from.
        v. At this stage, you shouldn't really need the NAT connection anymore, so you can remove it or deselect both of its connection options, "connected" and "connect at power on" so that it is easily available and can be turned on whenever, versus having to add it whenever you may need to access the internet. I would say it is best practice to delete it so that it cannot be defaulted to and you can ensure that your local network is working correctly.

4. Configuring the Machines IP
    a. On your Client Machine:
        i. Open a terminal and run the command:
            1. *ip a*. If you get an outpout like the one below, then you are likely also getting "Connection issues" popups. This is because the vm is recognizing your vmnet, but doesn't have an address yet. To set one, open up the network settings of your OS and navigate to IPv4 tab. Select manual for the method and then input an address. It'll match you vmnet, like 192.168.x.y, x being what you set or assigned your vmnet, and y being

whatever you like, between 1 and 255. I chose 130 to start, but its personal preference.

```
steve@steve-VMware-Virtual-Platform:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:7e:de:ef brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.107.130/24 brd 192.168.107.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
```

2. Then, for the subnet mask, input the same number that is listed on the vmnet.

ii. Also on the client machine, you should edit the netplan configuration. See the below photo to see the format, and replace the addresses field with your own client IP, name server field with your own name server ip, and search with your own domains. Ens36 is my leftover from when I added a NAT connection to download vmtools, and isn't necessary but it isn't hurting anything.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/netplan/01-network-manager-all.yaml
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.107.130/24
      nameservers:
        addresses:
          - 192.168.107.129
        search:
          - lab
          - jmu.lab

    ens36:
      dhcp4: yes
```

b. On the Primary Server:

i. Open a terminal and run the command:

1. *ip a*. If you get a similar output as before, then follow the same steps as above to set the IP and Subnet.

2. However, for the DNS Server address, put the local 127.0.0.1, showing that this machine is a DNS Sever. Below should be the output after you add this to the settings:

```
steve@steve-VMware-Virtual-Platform:/etc/bind$ resolvectl status
Global
         Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub
       DNS Servers: 127.0.0.1

Link 2 (ens33)
    Current Scopes: DNS
         Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 127.0.0.1
       DNS Servers: 127.0.0.1
```

3. The below netplan may or may not be necessary, use it as a last resort if you are still unable to connect at the end.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/netplan/01-network-manager-all.y
aml
network:
  version: 2
  ethernet:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.107.129/24
    ens36:
      dhcp4: yes
```

c. Back on the Client Machine:

i. After setting the Primary Server IP, go back to the client and set the DNS in the same area of settings and put in the Primary Server address you just assigned.

ii. Try pinging the client vm from another machine. The primary server with a command like ping 192.168.107.130 to ping the client machine to check for connectivity.

iii. In a terminal, run the command *resolvectl status,* and you should see something like this below

```
steve@steve-VMware-Virtual-Platform:~$ resolvectl status
Global
         Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub

Link 2 (ens33)
    Current Scopes: DNS
         Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 192.168.107.129
       DNS Servers: 192.168.107.129
        DNS Domain: lab jmu.lab
```

d. On the Delegated Server
   i. Open a terminal and run:
      1. *Ip a*. For this machine, all you need to do is set the IP address and subnet mask as you did before. Try pinging both machines like you did before, and if the output is receiving packets from other machines, they are connected.

5. Setting up DNS on the VM
   a. Setting up a primary server.
      i. This server will be, as the name says, the primary server that will be used when querying DNS. I will use example.jmu.lab for setting up the DNS. A good name for this vm would be VM1.
      ii. First, you must add a zone. A zone in this instance would be "lab". To add a zone, you must enter */etc/bind/named.conf.local* and edit it.
         1. The syntax is seen below, and a db file is assigned to this zone.

```
//
// Do any local configuration here
//

zone "lab" {
        type master;
        file "/etc/bind/db.lab";
};
```

      iii. The db file does not automatically appear, hower making a copy of db.local into the name you gave it in the zone, mine being "db.lab". Now, nano into the newly created db file.
         1. With the db file, you see lines that start with an @. The first line contains a lot of info, the line with SOA (Start of Authority).

After the SOA, you need to edit the localhost part. For me, it would be lab. and root.lab.

2. Below that is the serial number, and this is an important part too. Every time you update this file, the serial number must be incremented too. It must only be incremented once if you make multiple changes before restarting bind9. The other numbers in the SOA aren't too important right now.
3. Next, you must edit the lines at the bottom. At a minimum, you need an example IN A 192.168.x.y, x being the network octet and y being an octet of your choosing.
4. To make the primary server an authoritative server, @ IN NS ns.<domain>.lab. line is needed.
5. and a ns IN A 192.168.x.y line, the x being your network ip and the y octet being whatever you like, but stay away from the ips you have reserved for the Client and DNS servers.
6. To make this server authoritative, the jmu.lab. and ns1.jmu.lab. lines are required, with the ns1.jmu.lab. line ending with the delegated server's IP address.
7. I haven't edited the top comment, and it might be misleading but that is the comment from db.local, please disregard.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/bind/db.lab
;
; BIND data file for local loopback interface
;
$TTL    604800
@       IN      SOA     ns1.lab. root.lab. (
                              5         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.lab.

ns1     IN      A       192.168.107.129

jmu.lab.        IN      NS      ns1.jmu.lab.
ns1.jmu.lab.    IN      A       192.168.107.128
bc      IN      A       192.168.107.1
computer        IN      A       192.168.107.2
```

8. When adding to the domain server, you will likely run into errors. I these are after changing named.conf file, run a command to check for those errors: named-checkconf. I you are having issues with a zone or named-checkconf return ok, the try named-checkzone <zone-name> /etc/bind/db.<zone-name>. If it tells you an error, correct it and and keep trying till you get an OK.

```
steve@steve-VMware-Virtual-Platform:/etc/bind$ named-checkzone jmu.lab db.jmu.la
b
zone jmu.lab/IN: loaded serial 6
OK
```

9. At this point, running a dig command at the primary server is a good idea. Using the domain you made above, run a dig command like so:
    a. *dig @<primary server ip> example.jmu.lab* (my example domain). As you can see below, the status area shows whether or not querying the server was successful. You should see NOERROR, but you might see SERVFAIL or NXDOMAIN

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 computer.lab

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.107.129 computer.lab
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 50660
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 77f6f6477cf686730100000069a5d81f095a8d9a76c0eb43 (good)
;; QUESTION SECTION:
;computer.lab.                  IN      A

;; ANSWER SECTION:
computer.lab.           86400   IN      A       192.168.107.2

;; Query time: 1 msec
;; SERVER: 192.168.107.129#53(192.168.107.129) (UDP)
;; WHEN: Mon Mar 02 13:34:07 EST 2026
;; MSG SIZE  rcvd: 85
```

10. SERVFAIL likely means there is an error in your network configuration, but running the check discussed above in 4.c.iv on the server could prove useful.

11. NXDOMAIN means that the was no domain found (Non-existent Domain), maybe there was a typo, or once again run the check discussed above in 4.a.iii.8 on the server could prove useful.

b. Setting up the Secondary Server
   i. For setting up the secondary server, you follow the same steps.
   ii. First edit /etc/bind/named.conf.local to be like this for the delegated zone, .jmu in this case:

```
//
// Do any local configuration here
//

zone "jmu.lab" {
        type master;
        file "/etc/bind/db.jmu.lab";
};
```

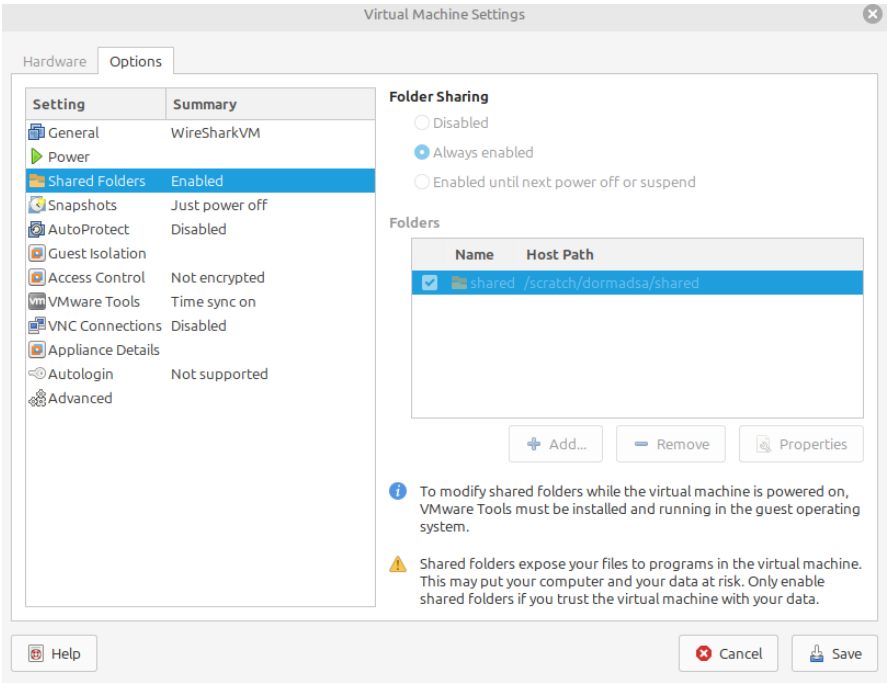
   iii. In this db file, the name space record points towards the server itself, and the other records are viable domains for <name>.jmu.lab.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/bind/db.jmu.lab
;
; BIND data file for local loopback interface
;
$TTL    604800
@       IN      SOA     ns1.jmu root.jmu (
                              6         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;

@       IN      NS      ns1.jmu.

ns1     IN      A       192.168.107.128
example IN      A       192.168.107.50
shea    IN      A       192.168.107.69
john    IN      A       192.168.107.67
alley   IN      A       192.168.107.68
king    IN      A       192.168.107.66
main    IN      A       192.168.107.65
quad    IN      A       192.168.107.64
```

6. Creating a shared folder between the VM and Host machine
    a. https://community.broadcom.com/vmware-cloud-foundation/communities/community-home/digestviewer/viewthread?GroupId=7171&MessageKey=e83f0009-629f-42a2-b0d0-a9f6ea56b3d0&CommunityKey=fb707ac3-9412-4fad-b7af-018f5da56d9f
    b. First, enter the setting of the VM you would like to remove information from. Configure the settings while the VM is off, the Screenshow below was taken when the VM was on. As you can see below, switch over to the options tab, then go down to “Shared Folders”. Select Always enabled at the top, then “+ Add”. You can then set a name for the shared folder, as well as set the location you want it saved to on the host machine. I liked to keep it clean and make the name the same for both folders.



c. Then, start up the VM. Open a terminal and enter the command below. This will fuse the hgfs directory with the host, allowing you to then view your shared folder in the /mnt/hgfs directory. Now, you can copy or move files through the command line into this directory and they will be put into the location you specified in the previous step on the host machine.

```
steve@steve-VMware-Virtual-Platform:~$ sudo vmhgfs-fuse .host:/ /mnt/hgfs -o all
ow_other
steve@steve-VMware-Virtual-Platform:~$ ls /mnt/hgfs
shared
```

7. Capturing Traffic with Wireshark

a. I kept getting ntp.ubuntu network traffic, caused by my isolated network wanting to access the outside. To stop this, I ran sudo timedatectl set-ntp off to stop this bloat from coming through. This was sending hundreds of pings every couple of seconds, making it hard to filter out so I disabled it.

8. Configuring DNSSEC

a. https://documentation.ubuntu.com/server/how-to/networking/install-dnssec/

b. Keys are in /var/cache/bind

c. Getting started – Parent

i. Now that DNS is working, it is time to implement DNSSEC. As we will need to write to the disc, we have to move to another directory. I moved to /var/lib/bind/ to implement DNSSEC.

ii. Firstly, copy over db.lab file from /etc/bind/ to /var/lib/bind/. You will need to edit it like so: We are going to add some delegation code here too, we are saving a step for later.

```
$TTL    86400
lab.    IN      SOA     lab. root.lab. (
                              3         ; Serial
                          43200         ; Refresh
                            900         ; Retry
                        1814400         ; Expire
                           7200 )       ; minimum

lab.    IN      NS      ns.lab.
ns      IN      A       192.168.107.129

jmu     IN      NS      ns.jmu.lab.
ns.jmu  IN      A       192.168.107.128
; bc    IN      A       192.168.107.1
computer        IN      A       192.168.107.2
```

iii. Still on the parent machine, nano into /etc/bind/named.conf.options.

1. Outside of the options area, add a dnssec-policy like the one below. You need both a KSK and ZSK. The algorithm RSASHA256 can be swapped out for your algorithm of choice.

```
dnssec-policy rsa-policy {
        keys {
                ksk lifetime 365d algorithm RSASHA256 2048;
                zsk lifetime 60d algorithm RSASHA256 1024;
        };
};
```

iv. Then, edit /etc/bind/named.conf.local file field to point the "/var/lib/bind/db.lab". and add some additional lines, including the dnssec-policy <name> and inline-signing yes. (inline-signing automatically generates a db.lab.signed file...Maybe change and mess with this for RRSig records)

v. Restart bind on this machine.

vi. Run the command dig @127.0.0.1 lab DNSKEY +dnssec +short to just output the KSK for this zone. You will copy this key and paste it to the client machine. These will be carried into the next step.

d. On the client machine

i. Create a file to hold the trust anchor. I called mine lab-trust.key. I added it to “etc/bind/lab-trust.key”. In the file, I added what is seen below. As seen below, the 257 recognizes this key as being the KSK for the lab zone. The base 64 key should be pasted at the end, with this format: “base-64 key”;. The client must directly trust lab. as the top of the chain instead of a root server.

```
trust-anchors {
        lab. static-key 257 3 8 "AwEAAdyubupVYO97+QR5LxX3eIh8vxOI5bxfVz6ieZqMF19e
5Ul2SD+MyyY5iM1VRutYEpLmfbMDDuW3ZElJzN4BRP8huMuzRf8V4nAuB3poxIUs+kRywr4n6et46oCBF
7oWmcw792jAKKXFiuegwZ3s5NvrZufCckq5wzgaPvMxJLZLkalddzkc0KuvwAtmGsOTT7lkn3sx59JgWG
SYiL+3FNZFSZPlcDyw3QuIO1hN+Vq2NwuSR2LkOFX3miCKx4d07hr27TtP3LGJiBhcyMeJRT3i4bOBbCt
wGelA6xq5i4k1fg2+R2e+2VZtqk35GJsh8c0yqKoahz96ZCSrWCHm690=";
};
```

ii. Also, add the key file to your named.conf include path: <include_key>. This should be added near the other include paths in the file. See below:

```
include "/etc/bind/lab-trust.key";
```

iii. Then, to check the setup, dig @parent_ip <domain> A +dnssec. You should see and additional ad flag. If so,

e. On Child Server

i. Firstly, copy over db.lab file from /etc/bind/ to /var/lib/bind/. You will need to edit it like so:

```
$TTL    86400   ; 1 day
jmu.lab.        IN      SOA     ns.jmu.lab. root.jmu.lab. (
                             10         ; Serial
                          43200         ; Refresh
                            900         ; Retry
                        1814400         ; Expire
                           7200 )       ; minimum
;

jmu.lab.        IN      NS      ns.jmu.lab.

ns.jmu.lab.     IN      A       192.168.107.128
example IN      A       192.168.107.50
shea    IN      A       192.168.107.69
john    IN      A       192.168.107.67
alley   IN      A       192.168.107.68
king    IN      A       192.168.107.66
main    IN      A       192.168.107.65
quad    IN      A       192.168.107.64
```

ii. Then, in named.conf.local, edit /etc/bind/named.conf.local file field to point the "/var/lib/bind/db.lab". and add some additional lines, including the dnssec-policy <name> and inline-signing yes.

iii. Restart bind.

f. Back on the client

i. dig @child_ip <domain> A +dnssec. Yous should see an A record, RRSIG for A record (POSSIBLY KEYS?)

g. Back on Child Server

i. Generate the Delegation Signer on Child and give it to the parent to link them.

1. Run: dnssec-dsfromkey -2 Kjmu.lab.+008+<KEYID>.key that should produce something like: jmu.lab. IN DS 34939 8 2 <digest>

2. Then, copy this data and save it for the next step on the parent server.

h. On the Parent

i. In /var/lib/bind/db.lab, add the following line for the DS record:

```
$TTL    86400
lab.    IN      SOA     lab. root.lab. (
                              3         ; Serial
                          43200         ; Refresh
                            900         ; Retry
                        1814400         ; Expire
                           7200 )       ; minimum

lab.    IN      NS      ns.lab.
ns      IN      A       192.168.107.129

jmu     IN      NS      ns.jmu.lab.
ns.jmu  IN      A       192.168.107.128
; bc    IN      A       192.168.107.1
computer        IN      A       192.168.107.2
jmu IN DS 34939 8 2 9682051422A73107F882F98D04F47492ECA60648DC071428E43477AC7F1
42C2
```

i. EXTRA

   i. To get DNSKEY and RRSIG files, I ran a dig @server_address DNSKEY +dnssec

   ii. Running sudo rndc flush is not a bad idea to run if you run into errors with keys, especially if you end up regenerating keys.

Source: Ubuntu Server Documentation - https://documentation.ubuntu.com/server/how-to/networking/install-dns/ Date accessed: 1/20/2026

Bind9 Man Pages, Date accessed 1/20/2026

https://bind9.readthedocs.io/en/v9.20.17/chapter3.html

Resolvctl Man Pages, Date accessed: 1/26/2026

https://www.freedesktop.org/software/systemd/man/latest/resolvectl.html

Resolved Man Pages, Date accessed: 1/27/2026

https://www.freedesktop.org/software/systemd/man/latest/systemd-resolved.service.html